Python OOP – Constructor Practice Assignment

Topic: Classes, Objects & Constructors (`__init__`)

Objective

In this assignment, students will practice the basic concepts of **Object-Oriented Programming (OOP)** in Python.

Students will learn how to:

- Create Classes
- Create Objects
- Use Constructors (`__init__`)
- Initialize object attributes
- Create and use Methods
- Store data inside objects
- Use Lists inside Classes
- Perform operations using Methods
- Make objects interact with each other

Question 1 – Movie Class

Create a class named `Movie`.

Requirements

The class should have the following attributes:

- `title`
- `duration`
- `genre`

Use a **constructor (`__init__`)** to initialize these values.

Create a method:

```
check_duration()
```

This method should check whether the movie is **longer than 2 hours**.

Expected Behaviour

If the movie duration is greater than 120 minutes:

```
The movie is longer than 2 hours.
```

Otherwise:

```
The movie is 2 hours or less.
```

Example

```
Title: Avengers
Duration: 150 minutes
Genre: Action

The movie is longer than 2 hours.
```

Concepts Covered

- Class
- Object
- Constructor
- Instance Attributes
- Method
- Conditional Statement

Question 2 – Restaurant Class

Create a class named `Restaurant`.

Requirements

The class should have:

- `name`
- `cuisine_type`

Use a **constructor (** `__init__` **)** to initialize these values.

Create the following methods:

1. `describe_restaurant()`

This method should display the restaurant's name and cuisine type.

2. `open_restaurant()`

This method should print:

```
The restaurant is open.
```

Example

```
Restaurant Name: Spice Hub
Cuisine Type: Indian

Restaurant: Spice Hub
Cuisine: Indian
The restaurant is open.
```

Concepts Covered

- Class
- Object
- Constructor
- Instance Attributes
- Multiple Methods

Question 3 – Smartphone Class

Create a class named `Smartphone`.

Requirements

Use a constructor (`__init__`) to initialize:

- `brand`
- `model`

Create the following methods:

1. `call()`

Display a message that the phone is making a call.

2. `play_music()`

Display a message that the phone is playing music.

3. `install_app()`

Display a message that an application has been installed.

Example

```
Brand: Samsung
Model: Galaxy S24

Calling...
Playing music...
App installed successfully.
```

Concepts Covered

- Constructor
- Object Attributes
- Methods
- Object Creation
- Method Calling

Question 4 – Online Order Management System

Create a simple **Online Order Management System** using Python OOP.

In this project, students need to create **three classes**:

```
Product
Customer
Order
```

Class 1 – Product

The `Product` class should store product information.

Attributes

- `product_name`
- `price`

Use a constructor (`__init__`) to initialize these values.

Example

```
Product Name: Laptop
Price: ₹55000
```

Class 2 – Customer

The `Customer` class should store customer details.

Attributes

- `name`
- `email`
- `phone`

Use a constructor (`__init__`) to initialize these values.

Example

```
Name: Adnan
Email: adnan@example.com
Phone: 9876543210
```

Class 3 – Order

The `Order` class should manage the customer's order and cart items.

Attributes

The class should contain:

- Customer information
- Cart/List of products

Use a constructor (`__init__`) to initialize the required values.

The cart should initially be an empty list.

Example:

```
self.cart = []
```

Required Methods in Order Class

1. `add_product()`

This method should add a product to the cart.

Example:

```
Laptop added to cart.
Mobile added to cart.
Mouse added to cart.
```

---

2. `calculate_total()`

This method should calculate the total price of all products present in the cart.

For example:

```
Laptop  → ₹55000
Mouse   → ₹1000
Keyboard → ₹2000

Total = ₹58000
```

---

3. `print_receipt()`

This method should display the final receipt.

The receipt should contain:

- Customer details
- Product names
- Product prices
- Total bill

Example Output

```
================================
           ORDER RECEIPT
================================

Customer Name: Adnan
Email: adnan@example.com

Products:
Laptop      ₹55000
Mouse       ₹1000
Keyboard    ₹2000

--------------------------------
```

```
Total Bill: ₹58000
================================
```

Project Requirements

The program should allow the user to:

1. Create products
2. Create a customer
3. Create an order
4. Add products to the cart
5. Store products inside a list
6. Calculate the total bill
7. Display the final receipt

Concepts Students Must Use

Students must use the following OOP concepts:

1. Classes

Create:

```
class Product:
```

```
class Customer:
```

```
class Order:
```

2. Constructor

Use:

```
def __init__(self):
```

to initialize object data.

3. Objects

Create objects from the classes.

Example:

```
product1 = Product(...)
customer1 = Customer(...)
order1 = Order(...)
```

4. Lists in Classes

The `Order` class should maintain a list of products:

```
self.cart = []
```

5. Methods

Create methods to perform different operations.

6. Loops

Use loops to display all products in the cart and calculate the total.

7. Object Interaction

The `Order` object should work with:

- `Customer` objects
- `Product` objects

Bonus Challenge ⭐

After completing the basic project, students can add:

- Remove product from cart
- Display number of products
- Apply discount
- Calculate GST
- Generate final amount after discount and GST

Example:

```
Subtotal: ₹58000
Discount: ₹2000
GST: ₹10080
Final Amount: ₹66080
```

Submission Requirement

Students should submit:

- Python `.py` file
- Proper class and method names
- Constructor implementation
- Proper indentation
- Sample output

Note: Focus on understanding how constructors initialize object data and how different objects can interact with each other.